

accumulate 정리

범위 안의 값을 누적하여 합계나 누적 결과를 만드는 알고리즘입니다.

사용처 (Where)

- 숫자 합계
- 초기값을 가진 누적 계산
- 문자열 연결 또는 곱 계산

방법 (How to Use)

- numeric 헤더 포함
- 범위와 초기값을 넘김
- 초기값 타입이 결과 타입을 결정
- 사용자 연산을 넘겨 응용 가능

기본 정보

- 필요 헤더: `#include <numeric>`
- 기본 선언: `int sum = accumulate(v.begin(), v.end(), 0);`

왜 써야 할까? (Why)

- 합계를 구하는 반복문을 짧게 표현
- 초기값을 통해 타입 제어 가능
- 누적 로직을 함수처럼 표현할 수 있음

주요 함수

함수/문법	의미
<code>accumulate(first,last,init)</code>	기본 합계
네 번째 인자	사용자 누적 함수
<code>init</code>	초기값 및 결과 타입

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <numeric>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4};
    int sum = accumulate(v.begin(), v.end(), 0);
    cout << sum;
}
```

accumulate 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
accumulate	O(n)
추가 메모리	거의 없음
문자열 누적	길이에 따라 비용 증가

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	accumulate 함수 호출만 알면 됨	반복문과 조건을 직접 설계	qsort/직접 반복문 사용
코드 길이	짧고 의도가 분명함	길어지고 실수 지점 증가	자료형 처리와 캐스팅이 번거로움
안정성	표준 라이브러리라 검증됨	구현 실수 가능	포인터/범위 실수 가능
추천 상황	대부분의 일반 상황	원리를 공부하거나 특수 최적화	C 코드와 연동할 때

응용: 평균 점수 계산

vector의 합계를 구한 뒤 size로 나누어 평균을 계산합니다.

```
#include <iostream>
#include <vector>
#include <numeric>
using namespace std;

int main() {
    vector<int> score = {80, 90, 100};
    int sum = accumulate(score.begin(), score.end(), 0);
    double avg = (double)sum / score.size();

    cout << avg;
}
```

처음 배울 때 자주 하는 실수

- 초기값을 0으로 주면 int 계산이 됨. 큰 수는 0LL 사용
- 빈 컨테이너의 평균 계산은 size 0 확인
- numeric 헤더를 빼먹기 쉬움

비슷한 항목과 차이

- count는 개수
- max_element는 최댓값 위치
- reduce는 병렬/순서 차이가 있을 수 있는 누적

한 줄 정리: `accumulate`은(는) '숫자 합계' 상황에서 먼저 떠올리면 좋은 STL입니다.